

Project Report

Name: Praveer Raj

Branch: Maths and Computing

Reg. No.: 230957002

Roll No.: 1

Project Title:-

Movie Recommender System

Abstract

This project presents a content-based movie recommender system that helps users find movies similar to their selected preferences. Utilizing a machine learning model trained on a dataset of movies with associated metadata, the system computes similarity scores based on movie features. Once a user selects a movie, the application displays the top five similar movies along with their posters using The Movie Database (TMDb) API. The project uses Python, Streamlit for the web interface, and Pickle for loading precomputed data structures. The recommender system delivers fast and visually interactive recommendations. This project demonstrates the power of combining machine learning with simple web deployment tools to build scalable and efficient recommendation engines.

Introduction

Problem Statement: Finding relevant movies based on user preferences is a challenge given the vast amount of content available. Users often waste time browsing through platforms. The recommender system aims to automate this process by suggesting movies similar to a selected title.

Objectives:

- To build a web-based content-based movie recommender system.
- To enhance user experience by recommending visually engaging movie options.
- To learn and apply machine learning techniques for similarity measurement.

Scope: The system is designed to work with a static dataset of movies and their similarity scores. It integrates real-time data from TMDb for poster retrieval. The recommendations are limited to top five similar movies per selected title.

Dataset and Preprocessing

Dataset Description: The dataset used includes movie titles, IDs, and feature vectors required for computing similarities. It was preprocessed and stored using Python's Pickle module in the files movie_list.pkl and similarity.pkl.

Exploratory Data Analysis (EDA): Basic checks and exploration were performed to ensure data integrity and usefulness. Feature vectors were generated using textual metadata (e.g., genre, keywords, cast, crew).

Preprocessing Steps:

- Textual data was combined and vectorized using techniques like CountVectorizer or TF-IDF.
- Similarity scores were computed using cosine similarity.
- Final data structures were serialized using Pickle.

Methodology

Machine Learning Model(s) Used: No supervised model was used; the system relies on content-based filtering. Cosine similarity was the core metric to compare movie vectors.

Justification: Content-based filtering does not require user ratings and works well for new or limited-interaction systems.

Implementation Details:

- Movie vectors were created based on features.
- Cosine similarity was calculated and stored.
- The Streamlit interface allows selection of a movie title and displays similar movies using the TMDb API.

Experimental Setup

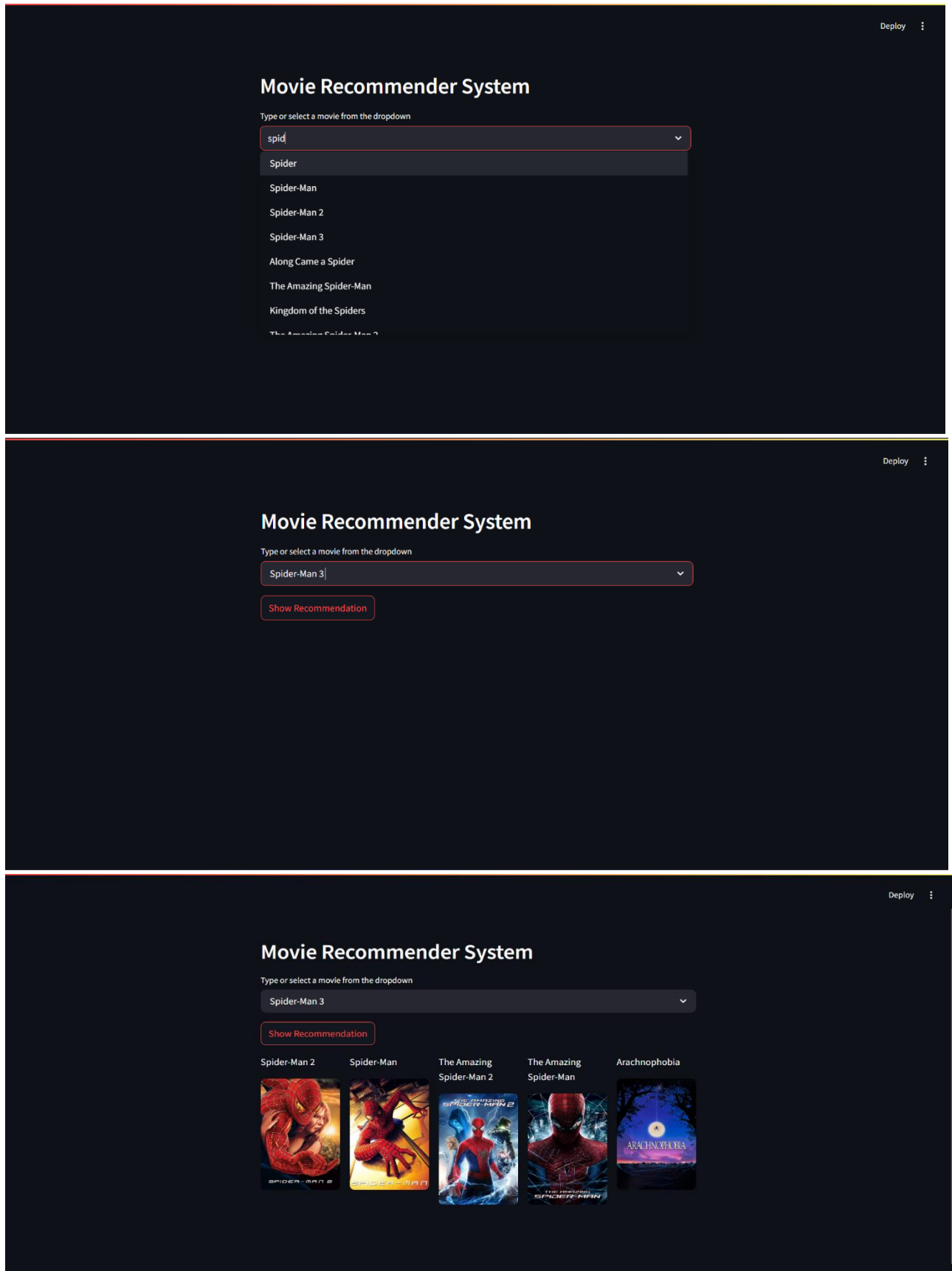
Hardware/Software Used:

- Python 3.8+
- Streamlit
- Pickle
- Requests library
- TMDb API

Hyperparameters: As this is not a training-based system, no hyperparameters like learning rate or epochs are involved.

Train-Test Split: Not applicable, as there is no training phase.

Results and screenshots of UI :-



Performance Metrics: Since this is not a classification/regression task, traditional metrics like accuracy or F1-score are not applicable.

Comparison of Models: Not applicable, only one approach (cosine similarity) was used.

Visualization of Results: Posters of the top 5 recommended movies are displayed in a responsive UI using Streamlit columns.

Error Analysis: Some movie recommendations may appear less relevant due to limitations in metadata or missing poster data from TMDb.

Conclusion & Future Work

This project successfully built a working content-based movie recommender system using cosine similarity. The system is responsive, simple, and visually appealing. In the future, hybrid methods combining collaborative filtering and deep learning can be explored. Incorporating user profiles and feedback mechanisms would also enhance recommendation quality.

References

- TMDb API Documentation. Available at: <https://developers.themoviedb.org/3>
- Python Official Documentation. Available at: <https://docs.python.org/3/>
- Streamlit Official Documentation. Available at: <https://docs.streamlit.io/>
- Several AI models

Appendix

- Code snippets for recommend() function and fetch_poster() API integration.

```
# Function to fetch movie poster using TMDb API
import requests

def fetch_poster(movie_id):
    url = f"https://api.themoviedb.org/3/movie/{movie_id}?api_key=YOUR_API_KEY&language=en-US"
    data = requests.get(url).json()
    poster_path = data['poster_path']
    full_path = "https://image.tmdb.org/t/p/w500/" + poster_path
    return full_path
```



Recommendation logic based on cosine similarity

```
def recommend(movie):  
    index = movies[movies['title'] == movie].index[0]  
    distances = sorted(list(enumerate(similarity[index])), reverse=True, key=lambda x: x[1])  
    recommended_movie_names = []  
    recommended_movie_posters = []  
    for i in distances[1:6]:  
        movie_id = movies.iloc[i[0]].movie_id  
        recommended_movie_posters.append(fetch_poster(movie_id))  
        recommended_movie_names.append(movies.iloc[i[0]].title)  
  
    return recommended_movie_names, recommended_movie_posters
```